BGKIT: Reading Compressed Context through a Language Model's Embedding Layer

Gabriel Pickard

October 2026

Abstract

BGKIT gives a language model a common interface for reading compressed context. A small bidirectional encoder reads a source under a prompt describing the information need and produces a chosen number of vectors in the decoder's input embedding space, which the decoder reads in slots alongside ordinary text; the same interface carries documents, tool results, conversation histories and persistent repository overviews. With a 350M-parameter encoder and decoder trained on a single GPU, the encoder's vectors prove far more useful than the tokens at the very positions they occupy: at identical retained positions they score 0.74 against 0.26 on BABILong 16k, 0.64 against 0.31 on HotpotQA and 0.30 against 0.07 on LongMemEval-S. With the decoder held fixed, the compressed context also surpasses the full, uncompressed text on BABILong, HotpotQA and LoCoMo. On nine long-context benchmarks at fixed decoder budgets it significantly outperforms truncation in 16 of 26 settings and LongLLMLingua in 13 of 22, and exceeds BM25 retrieval where answers integrate facts spread across the context, in several cases with a quarter of BM25's positions; question-focused summaries from a 35B model remain stronger on some multi-hop questions, and exact strings are not retained (RULER). At 110k tokens the decoder needs about 50× less memory beyond its weights and decodes 4× faster, while encoding costs roughly one full-context prefill.

1 Introduction

A language model can condition only on what fits in its context window. Retrieved documents, tool output, earlier conversation and source code compete for the same positions, and every position adds to the cost of each forward pass. Vision–language models handle a similar problem for images: a separate encoder turns many pixels into a few vectors, which are projected into the language model's token-embedding space and read like tokens [1, 12].

BGKIT applies the same principle to text. An encoder reads a text together with a prompt that states what the reader needs from it, for example *"answer: who founded the company?"* or *"lines 80–140 of* `io.py`*: where is the buffer flushed?"*, and emits a short sequence of vectors in the decoder's input embedding space. The decoder reads them in a slot in place of the text (Figure 1). The compression ratio is chosen per slot at inference time, and an input can hold any number of slots alongside ordinary text. Because the slots are vectors in the decoder's own embedding space, the encoder can also read its own output, which allows directory trees and long histories to be summarised hierarchically. An encoding produced under a question-independent prompt can, moreover, remain in context as a persistent, compressed view of a repository, a memory or a long history, without consuming the context window on raw text. BGKIT is intended as a building block for this kind of ambient awareness beyond the context window.

The paper makes three contributions:

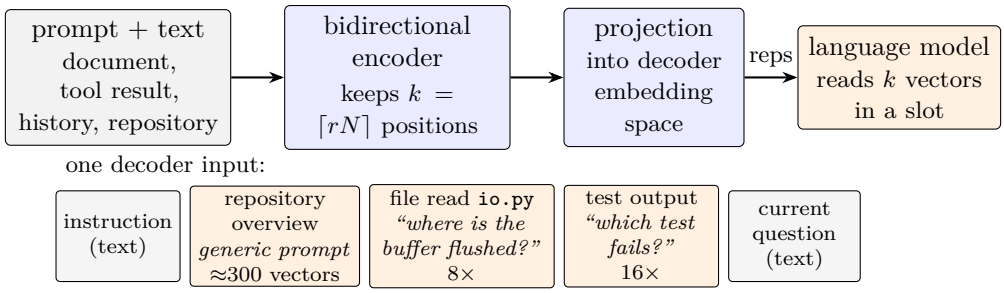


Figure 1: The BGKIT interface. Top: an encoder reads a text under a prompt and emits k vectors ("reps") in the decoder's embedding space. Bottom: the decoder reads them in slots among ordinary tokens; each slot has its own source, prompt and budget, and one input can hold any number of them.

- **A practical interface** (Sections 2–3): prompt-conditioned context vectors with a budget chosen per slot, any number of slots mixed with text, and one set of weights serving documents, tool results, histories and hierarchical overviews; with its training recipe and an agent harness in which every tool result arrives compressed.
- **An empirical insight** (Sections 5–6): at the encoder's own retained positions its vectors are far more useful than the tokens there, and with the decoder held fixed the compressed context can surpass the full text; we measure this against truncation, BM25 retrieval, LongLLMLingua and summaries on nine benchmarks, with paired intervals.
- **Costs and a reusable implementation** (Sections 7–9): the effect of the compression ratio, end-to-end cost, an agent on SWE-bench Lite, web research and conversational memory, and released weights with standalone inference code.

2 Method

Encoder. The encoder is LFM2.5-Encoder-350M, a 16-layer bidirectional model with hidden size 1024. Its input is a prompt, a separator and the text to encode, so every text position can attend to the prompt.

Compression. After layer 6 of 16, a scoring head ranks the text positions, and the top $k = \lceil rN \rceil$ of the N positions are kept for a compression ratio r. A learned *survival marker* is added to the hidden state at each kept position, so the remaining layers know which positions will be passed on. These layers still run over the whole sequence, so each retained position can aggregate information from the entire text. The final hidden states at the kept positions form the encoding. Long texts are cut into chunks of up to 2,048 tokens, each encoded under the same prompt into its own slot. The ratio is a free parameter at inference time: the same weights produce 4×, 16× or 128× encodings.

Projection. A projection stage (an attention layer over the kept positions followed by an MLP) maps the encoding into the decoder's input embedding space, and each output vector is rescaled to a fixed norm, the mean row norm of the decoder's token embeddings. Earlier versions without a fixed interface exhibited representational collapse: the projected vectors converged on a shared direction while retaining a plausible norm.

Decoder. The decoder is LFM2.5-350M (instruction-tuned), and its input is rendered with its own chat template. Each slot is marked by a placeholder token that is replaced by the slot's vectors, and an input may contain any number of slots interleaved with text. A rank-32 low-rank adapter on the decoder's first eight layers is trained to read the slots. The adapter applies at all positions, so it also changes how the decoder reads ordinary text.

Hierarchical encodings. Because the vectors live in the decoder's embedding space, the encoder can also be applied to its own output. We use this in three places:

- *Repositories* are encoded bottom-up. Each file is encoded at 8×, and each directory encodes its children's vectors at a further 4×, up to a root *overview* of a few hundred vectors. Nodes are cached and can be opened with a `browse` tool.
- *Long histories* are compacted by folding older turns into cells encoded at 16×. Cells merge pairwise at 4×, so the compacted history grows roughly logarithmically.
- *Conversational memory* is a set of per-session encodings with a compact index of all sessions. A session can be re-read under a new prompt.

The evaluations below exercise the repository overview and the memory index. History compaction is not evaluated separately.

Agent harness. In our agent harness, file reads, code search, shell output, web search, page reads, find-in-page and memory lookups all return slots rather than text. Each slot is encoded under the agent's own statement of what it is looking for. Tools have fixed compression ratios: 4× for search hits and find-in-page, where answers are often copied verbatim; 8× for file reads; 16× for other search, shell output and page reads; 32× for the repository overview and compacted history.

3 Training

Objectives. The decoder is trained to produce a target (an answer, an agent's next turn, or a continuation of the text) from an input in which the context appears only as encoded vectors. Three further signals are added to this cross-entropy loss:

- *Distillation from a full-text reader.* A decoder with a separately trained adapter reads the same input with the context as plain text, and the model reading encodings is trained to match its next-token distribution.
- *Prompt contrast.* A document is also encoded under a different question about it, and the correct answer is trained to be more likely under the right prompt than under the wrong one.
- *Preservation.* On a stream of ordinary chat and long-text reading, the adapted decoder is distilled towards the original decoder, to limit degradation of its general behaviour.

The scoring head receives a small auxiliary loss that encourages it to match the full-text reader's attention from the answer to the context.

Compression curriculum. Each example's ratio is sampled between 4× and a floor that moves from 64× towards 128× during training. In agent data, tool slots use the harness ratios, randomly scaled by up to a factor of two.

Data. Training mixes reconstruction and continuation of plain text (web text, code, books, dialogue) with task data rendered as conversations whose contexts become slots:

- reading comprehension and long-document QA, including the training splits of HotpotQA, QuALITY and NarrativeQA (benchmark-test stories excluded);
- bAbI tasks 1–3 (training split) hidden in book, web and wiki filler, which is how BABILong builds its test set;
- multi-hop QA with distractors and synthetic needle retrieval;
- agent trajectories on software repositories (generated by a larger teacher model and rejection-sampled), with repository QA, file localisation and repository overviews;
- web-research trajectories mapped onto the web tools;
- multi-session conversations (from MSC, Chronicles and SHARE) with generated questions about their history.

No evaluation item appears in training. BABILong, HotpotQA, QuALITY and NarrativeQA are therefore in-distribution task formats; MuSiQue, RULER, LongMemEval and LoCoMo were used only for evaluation.

Parameters and compute. In the final stages the base encoder weights are frozen. A rank-16 adapter on its top four layers, the scoring head and the projection are trained, together with the decoder adapter. All training ran on a single NVIDIA DGX Spark. The evaluated checkpoint ends a chain of warm-started development runs: its final three stages took 40,000 steps (8,000, 12,000 and 20,000), with the agentic, web and memory data at full weight in the last 20,000. The encoder and adapters inherit weights from earlier runs on overlapping data.

4 Evaluation setup

Every benchmark comparison uses a fixed *budget*: the number of decoder positions the context may occupy. The arms are:

bgkit The context is compressed to fit the budget, with the question as the encoder prompt.

text The same trained decoder reads the context as text, middle-truncated to the budget.

base The original decoder (adapter off) reads the same truncated text.

BM25 Retrieval at the budget: the context is cut into passages of at most 128 tokens, the passages are ranked against the question with BM25, and the best ones are kept up to the budget, in document order.

LongLLMLingua Question-aware token pruning [8] to the budget, with the method's recommended settings and Qwen3-1.7B as the scoring model (five times the size of our encoder); the result is cut to the budget in the decoder's tokens.

summary Qwen3.6-35B-A3B writes a question-focused summary at the budget (map-reduce over 12k-token chunks for longer contexts).

base, full context The original decoder reads the whole context without truncation. This arm is not budget-matched.

The trained decoder was trained to read compressed slots, not to answer from text, so its reading of plain text may be weaker than the original decoder's. The three question-aware text baselines (BM25, LongLLMLingua, summary) were therefore read by both decoders, and each cell reports the better of the two; the *text* and *base* columns report truncated text under each decoder.

BABILong [11] (tasks qa1 and qa2: 66 and 34 items per length), RULER [6] and QuALITY [18] are scored by accuracy. NarrativeQA [10], HotpotQA [23], MuSiQue [20], LongMemEval [21] and LoCoMo [15] are scored by an LLM judge (Qwen3.6-35B-A3B with LongMemEval's yes/no judging prompt; a 4-bit GGUF build for the arms of the original sweep and an NVFP4 build for the baselines added later; an FP8 build agreed with the original on 99% of 1,500 sweep verdicts). All arms are scored on the same items. Cells have 60–100 items; we compare arms by the mean per-item difference with a 95% paired-bootstrap interval (10,000 resamples), given for every cell in Appendix A, and call a difference significant when the interval excludes zero.

5 Long contexts at a fixed budget

Table 1 reports the nine benchmarks. Contexts range from 1.4k tokens (HotpotQA) to 111k (LongMemEval-S), and budgets from 256 to 8,192 positions, so the compression ratio ranges from under 2× to about 54×.

Benchmarks not used in training. MuSiQue, RULER, LongMemEval and LoCoMo were used only for evaluation. On MuSiQue the compressed context scores 0.30–0.33, against 0.08–0.16 for truncated text, 0.17–0.19 for BM25 and 0.13 for the original decoder reading the entire context. On LoCoMo it scores 0.28, against 0.09–0.16 for truncated text and 0.19 for the entire context, and on LongMemEval-S (111k tokens per history) 0.20–0.30 against 0.15–0.18 for truncated text. On these two memory benchmarks BM25 retrieval performs comparably (0.27–0.34; no difference is significant). RULER is the exception, discussed below.

Against text baselines at the same budget. Appendix A gives the paired difference for every cell. The compressed context scores significantly above truncated text in 16 of 26 cells and above LongLLMLingua in 13 of 22, and is significantly below either only on RULER (LongLLMLingua at 2,048 positions). Against BM25 it is significantly above on BABILong (by 0.29–0.50), HotpotQA (0.14–0.25) and MuSiQue (0.11–0.15), within noise on QuALITY, NarrativeQA, LongMemEval-S and LoCoMo, and below on RULER. Its advantage is concentrated on questions that require integrating facts distributed across the context: bAbI's chains of movements and multi-hop questions spanning several paragraphs. Question-focused summaries from the 35B model, whose generation is far more expensive than a single pass of the 350M encoder, score higher on MuSiQue (0.43 against 0.30–0.31, significant at 512 positions) and on RULER, comparably on HotpotQA and QuALITY, and lower on BABILong 4k (0.25–0.32 against 0.79–0.81).

Effective budget. Table 2 compares the compressed context at each benchmark's smallest budget with the best text arm run at the largest, four times as many positions (twice for LoCoMo). With a quarter of the positions, the compressed context scores significantly higher on BABILong, HotpotQA and MuSiQue (for example 0.79 in 512 positions against 0.38 for BM25 in 2,048 on BABILong 4k), comparably on QuALITY, NarrativeQA, LongMemEval-S and LoCoMo, and lower on RULER.

Benchmark (n)	Budget	bgkit	text	base	BM25	LongLLMLingua	summary	base, full context
BABILong 16k (100)	1,024	**0.74**	0.12	0.08	0.40	0.09	–	0.18
15.4k tokens	2,048	**0.76**	0.19	0.13	0.43	0.15	–	
	4,096	**0.74**	0.22	0.13	0.45	0.27	–	
BABILong 4k (100)	512	**0.79**	0.20	0.16	0.39	0.13	0.32	0.27
3.6k tokens	1,024	**0.81**	0.26	0.23	0.42	0.21	0.25	
	2,048	**0.88**	0.33	0.21	0.38	0.31	–	
RULER 4k (100)	512	0.20	0.09	0.07	**0.42**	0.02	0.41	0.58
3.7k tokens	1,024	0.20	0.15	0.11	**0.49**	0.18	0.34	
	2,048	0.17	0.22	0.24	**0.54**	0.39	–	
QuALITY (100)	512	0.36	0.31	0.25	0.37	0.30	**0.40**	0.31
6.0k tokens	1,024	**0.36**	0.33	0.27	0.33	0.28	–	
	2,048	0.37	0.34	0.26	0.33	0.37	**0.48**	
NarrativeQA (60)	2,048	**0.17**	0.07	0.05	0.13	0.12	–	–
58.6k tokens	4,096	**0.17**	0.10	0.10	0.15	–	–	
	8,192	**0.15**	0.12	0.08	0.13	–	–	
HotpotQA (100)	256	**0.64**	0.23	0.25	0.39	0.36	0.57	0.45
1.4k tokens	512	**0.62**	0.35	0.27	0.46	0.34	0.53	
	1,024	**0.60**	0.37	0.39	0.46	0.41	–	
MuSiQue (100)	512	0.30	0.08	0.08	0.19	0.10	**0.43**	0.13
2.5k tokens	1,024	0.31	0.08	0.10	0.17	0.15	**0.43**	
	2,048	**0.33**	0.16	0.10	0.18	0.17	–	
LongMemEval-S (60)	2,048	0.30	0.18	0.10	**0.32**	0.27	–	–
110.7k tokens	4,096	0.23	0.15	0.05	**0.33**	–	–	
	8,192	0.20	0.15	0.02	**0.27**	–	–	
LoCoMo (100)	1,024	0.28	0.09	0.08	**0.34**	0.18	–	0.19
18.8k tokens	2,048	0.28	0.16	0.13	**0.30**	0.24	–	

Table 1: Accuracy by benchmark and budget (decoder positions available to the context); the second line of each benchmark gives its mean context length. BABILong, RULER and QuALITY: accuracy; the others: LLM-judge accuracy. Text: the trained decoder reading middle-truncated text; base: the original decoder reading it. BM25, LongLLMLingua and summary: the better of the two decoders. LongLLMLingua was run at one budget for NarrativeQA and LongMemEval-S; summaries exist for the budgets shown (QuALITY: 60 and 56 items). Bold: best budget-matched arm. The last column is the original decoder reading the whole, untruncated context on the same items and is not budget-matched; it was not run for LongMemEval-S (contexts above the 32k-token limit of our full-context runs) and has too few shared items for NarrativeQA.

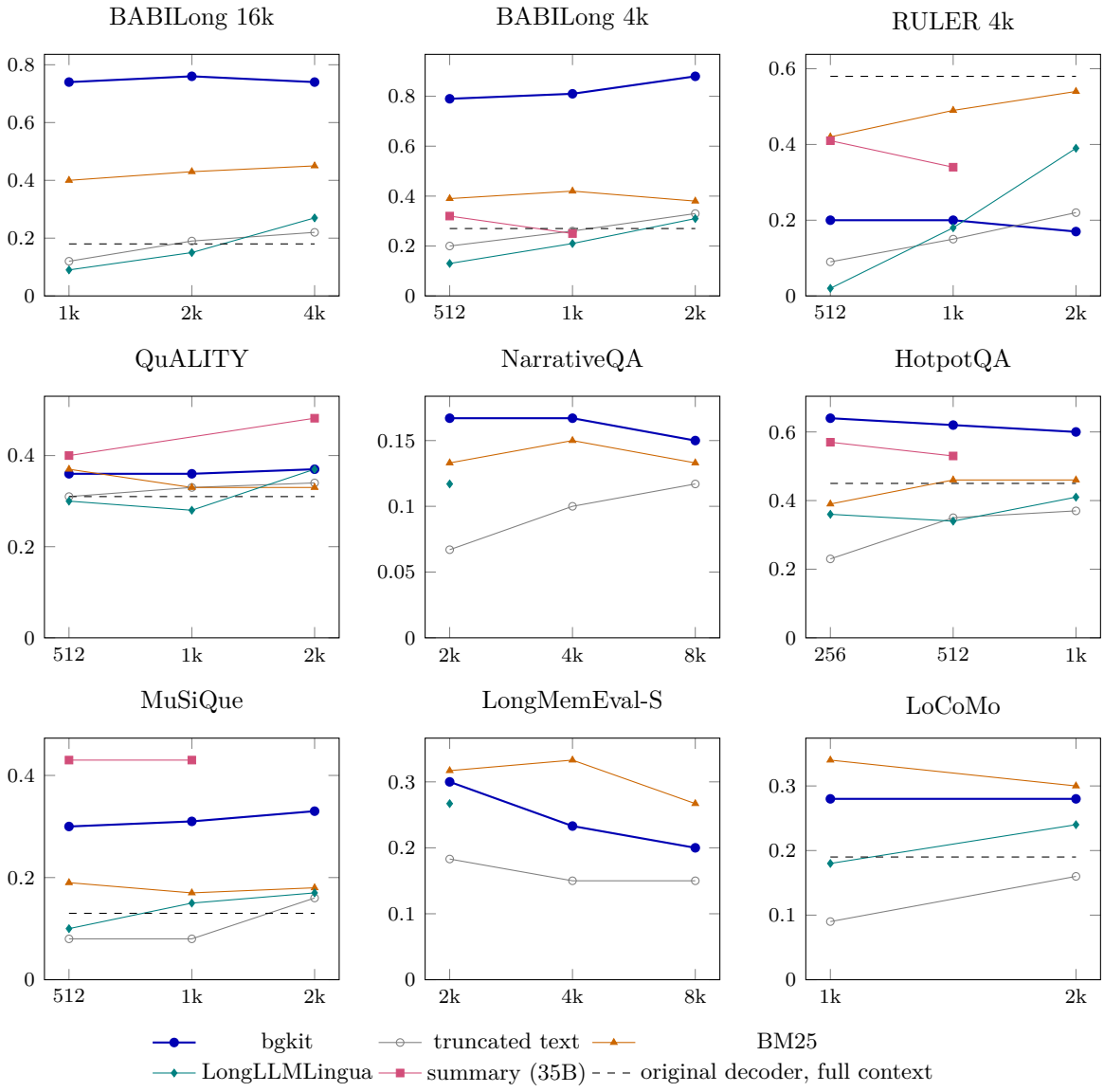


Figure 2: Score against budget (decoder positions for the context, log scale) for the arms of Table 1. Text baselines are read by the better of the two decoders. Dashed: the original decoder reading the whole context (not budget-matched).

The comparison is limited to the arms run at the larger budget: summaries exist there only for QuALITY, and on MuSiQue they already score higher at smaller budgets (0.43 at 512 positions; Table 1).

	bgkit		best text-only arm				
Benchmark	budget	score	budget	arm	score	positions	difference
BABILong 16k	1,024	0.74	4,096	BM25	0.45	4×	+0.29 [+0.17, +0.41]
BABILong 4k	512	0.79	2,048	BM25	0.38	4×	+0.41 [+0.29, +0.53]
RULER 4k	512	0.20	2,048	BM25	0.54	4×	−0.34 [−0.44, −0.24]
QuALITY	512	0.36	2,048	summary	0.48	4×	−0.09 [−0.18, +0.00]
NarrativeQA	2,048	0.17	8,192	BM25	0.13	4×	+0.03 [−0.08, +0.15]
HotpotQA	256	0.64	1,024	BM25	0.46	4×	+0.18 [+0.07, +0.29]
MuSiQue	512	0.30	2,048	BM25	0.18	4×	+0.12 [+0.01, +0.23]
LongMemEval-S	2,048	0.30	8,192	BM25	0.27	4×	+0.03 [−0.08, +0.15]
LoCoMo	1,024	0.28	2,048	BM25	0.30	2×	−0.02 [−0.12, +0.08]

Table 2: The compressed context at a benchmark's smallest budget against the best text-only arm run at the largest budget (truncated text under either decoder, BM25, LongLLMLingua, or a summary where one was generated at that budget, each with its better decoder). Positions: how many times more decoder positions the text arm has. Difference: mean paired difference with 95% bootstrap interval; bold where it excludes zero.

Against the whole context. Table 3 compares the compressed context with the whole, untruncated text, read once by the same trained decoder, so that only the form of the input differs, and once by the original decoder. With the same weights, the compressed context scores significantly higher on BABILong (0.74–0.79 against 0.37–0.44), HotpotQA (0.64 against 0.36) and LoCoMo, which was not used in training (0.28 against 0.17); it performs comparably on MuSiQue (0.30 against 0.20, interval including zero) and QuALITY, and lower on RULER (0.20 against 0.45). Against the original decoder reading the whole context it is higher on BABILong, HotpotQA (0.64 against 0.45) and MuSiQue (0.30 against 0.13). On questions that integrate facts from across a text, the decoder thus extracts more usable information from the prompted encoding than from the text itself, while attending to a fraction of the positions. Two caveats: the trained decoder's adapter was trained on slots rather than text, which may handicap its reading of text, and the task formats of BABILong and HotpotQA were in training (for the original decoder, part of the margin comes from that training rather than from compression).

Exact strings. RULER's needle tasks ask for exact keys and numbers. The full context scores 0.58, BM25 0.42–0.54 and the compressed context 0.17–0.20: arbitrary strings are not preserved at these ratios, and methods that return the matching passage verbatim are better suited to such lookups. The agent harness accordingly reads search hits at only 4×.

6 What the compressed context carries

The encoder performs two functions: it selects which positions to retain, and it computes a representation at each retained position after the remaining layers have attended over the full text and the prompt. Two measurements disentangle them.

		bgkit		whole context		bgkit minus	
Benchmark	Context	budget	score	same decoder	original	same decoder	original
BABILong 16k	15.4k	1,024	0.74	0.37	0.18	+0.37 [+0.26, +0.48]	+0.56 [+0.45, +0.67]
BABILong 4k	3.6k	512	0.79	0.44	0.27	+0.35 [+0.23, +0.46]	+0.52 [+0.41, +0.62]
RULER 4k	3.7k	512	0.20	0.45	0.58	−0.25 [−0.36, −0.14]	−0.38 [−0.48, −0.28]
QuALITY	6.0k	512	0.36	0.34	0.31	+0.02 [−0.08, +0.12]	+0.05 [−0.07, +0.16]
HotpotQA	1.4k	256	0.64	0.36	0.45	+0.28 [+0.18, +0.38]	+0.19 [+0.09, +0.29]
MuSiQue	2.5k	512	0.30	0.20	0.13	+0.10 [−0.01, +0.21]	+0.17 [+0.06, +0.27]
LoCoMo	18.8k	1,024	0.28	0.17	0.19	+0.11 [+0.02, +0.20]	+0.09 [−0.01, +0.19]

Table 3: The compressed context at each benchmark's smallest budget against the whole, untruncated context read by the same trained decoder and by the original decoder (benchmarks whose contexts fit the 32k-token limit of the full-context runs). Differences: mean paired difference with 95% bootstrap interval; bold where it excludes zero.

Vectors against the tokens they sit on. In the *kept tokens* arm the encoder runs exactly as in the BGKIT arm, with the same question as prompt and the same budget, but the decoder receives the token embeddings at the kept positions, in document order, instead of the encoder's vectors (Table 4). The positions and their number are identical; only what occupies them differs. The vectors score significantly higher on seven of nine benchmarks: 0.74 against 0.26 on BABILong 16k, 0.64 against 0.31 on HotpotQA, 0.30 against 0.07 on LongMemEval-S; QuALITY and NarrativeQA are within noise. The kept tokens on their own score in the range of LongLLMLingua's question-aware token pruning at the same budget (higher on BABILong, lower on the two memory benchmarks). The advantage of the compressed context therefore resides primarily in the representations: a vector at a retained position encodes information from discarded text, which no selection of tokens can. Together with the survival marker, which tells the later layers which positions will be passed on, this suggests that the kept positions act as places where the encoder gathers information for the reader; the random-selection result below is consistent with some freedom in where that information is placed. We offer this as an interpretation of the measurements, not as a mechanistic account.

			bgkit selection				
Benchmark	Budget	Ratio	vectors	tokens	difference	LongLLMLingua	text
BABILong 16k	1,024	15×	0.74	0.26	+0.48 [+0.36, +0.60]	0.09	0.12
BABILong 4k	512	7×	0.79	0.32	+0.47 [+0.36, +0.58]	0.13	0.20
RULER 4k	512	7×	0.20	0.08	+0.12 [+0.05, +0.19]	0.02	0.09
QuALITY	512	12×	0.36	0.33	+0.03 [−0.05, +0.11]	0.30	0.31
NarrativeQA	2,048	29×	0.17	0.12	+0.05 [−0.05, +0.15]	0.12	0.07
HotpotQA	256	6×	0.64	0.31	+0.33 [+0.20, +0.46]	0.36	0.23
MuSiQue	512	5×	0.30	0.16	+0.14 [+0.04, +0.24]	0.10	0.08
LongMemEval-S	2,048	54×	0.30	0.07	+0.23 [+0.13, +0.35]	0.27	0.18
LoCoMo	1,024	18×	0.28	0.12	+0.16 [+0.08, +0.24]	0.18	0.09

Table 4: Same selection, different contents, at each benchmark's smallest budget. Vectors: the BGKIT arm. Tokens: the token embeddings at the same kept positions (better of the two decoders). Difference: vectors minus tokens, mean paired difference with 95% bootstrap interval; bold where it excludes zero. LongLLMLingua and text (truncated, trained decoder) at the same budget for scale.

A worked example. Figure 3 shows one BABILong item. The selection keeps all three facts about Sandra verbatim, in order, among the 1,027 positions kept from 14,793 tokens. Given those tokens, both decoders answer with an earlier location; given the encoder's vectors at the same positions, the decoder answers correctly. The difference is thus not which facts survive selection but what the vectors encode about them, here the order of Sandra's movements.

Question: Where is Sandra? **Answer:** garden

Context: 14,793 tokens of book text with bAbI facts inserted; compressed 14× into 1,027 positions in eight slots, with the question as the encoder prompt.

Facts about Sandra, in order, and the kept tokens within them:

"Sandra travelled to the bedroom."	all 6 tokens kept
"Sandra went to the kitchen."	all 6 tokens kept
"Sandra journeyed to the garden."	all 7 tokens kept

Kept tokens (trained or original decoder): "Sandra is in the kitchen."
Encoder vectors at the same positions: "garden"
Whole text, same decoder: "Sandra is in the garden." **BM25**, 1,024 positions: "Sandra is in the kitchen."

Figure 3: One BABILong 16k item (`babilong:16k:qa1:38`). Kept tokens and the facts' positions from `scripts/bgkit2_example_kept.py`; answers from the benchmark runs.

The prompt and the selection. Table 5 varies how the encoding is produced, on the items and checkpoint of Table 6. Encoding without the question lowers exact match from 0.58–0.90 to 0.04–0.21. With the prompt for a different question about the same document, the loss on the correct answer exceeds the no-context loss on SQuAD (3.16 against 2.30 at 4×) and multi-needle (4.32 against 3.01). Replacing the learned position selection with random positions of the same number changes exact match little on SQuAD (0.61 in both) and lowers it on multi-needle (0.54 against 0.58) and bAbI-in-filler (0.82 against 0.90). The content of the representations is thus determined chiefly by the conditioning prompt, and only secondarily by the choice of positions. The encoding is specific to its prompt: one made under one question can be reused for another only when the prompt does not depend on the question, as for the repository overview and the memory index.

	SQuAD			Multi-needle			bAbI-in-filler		
Encoder condition	EM	$\mathcal{L}_{4\times}$	$\mathcal{L}_{64\times}$	EM	$\mathcal{L}_{4\times}$	$\mathcal{L}_{64\times}$	EM	$\mathcal{L}_{4\times}$	$\mathcal{L}_{64\times}$
question prompt, learned selection	0.61	0.40	0.59	0.58	0.47	0.59	0.90	0.22	0.35
question prompt, random selection	0.61	0.43	0.59	0.54	0.51	0.67	0.82	0.20	0.61
no prompt	0.07	1.69	2.01	0.04	2.50	2.88	0.21	1.73	1.44
prompt for a different question	0.10	3.16	2.64	0.01	4.32	4.00	0.45	1.71	1.54
no context	–	2.30		–	3.01		–	3.48	

Table 5: Encoder conditions, same items and checkpoint as Table 6. EM: exact match of generated answers at 64×. $\mathcal{L}$: decoder loss on the gold answer (nats per token) at 4× and 64×; lower is better. "No context" is the decoder's loss with the context removed.

7 Compression ratio

Table 6 gives the decoder's loss on the gold answer at four compression ratios, on held-out items from three training task families and an earlier checkpoint of the same model. From 4× to 64× the loss rises by 0.12–0.19 nats, while removing the context costs 1.9–3.3 nats. At 128× the loss rises further on SQuAD and bAbI-in-filler (to 1.04 and 0.84 nats). The benchmarks show the same pattern where the budget varies the ratio: BABILong 16k scores 0.74–0.76 from about 4× to 15×, and NarrativeQA 0.15–0.17 from about 7× to 29×.

Task	4×	16×	64×	128×	no context
SQuAD	0.40	0.43	0.59	1.04	2.30
Multi-needle	0.47	0.58	0.59	0.66	3.01
bAbI-in-filler	0.22	0.14	0.35	0.84	3.48

Table 6: Decoder loss on the gold answer (nats per token) by compression ratio: held-out items of three training task families (96 each), earlier checkpoint, question as the encoder prompt.

8 Cost

Table 7 measures one question over a long context with the released model (bf16, one GPU, medians of three runs after a warm-up): the original decoder prefilling the whole context, against the encoder pass over the context in 2,048-token chunks followed by a decoder prefill over the slots. The decoder's cost scales with the number of positions it attends to. At 110k tokens, the memory the decoder needs beyond its weights drops from 5.0 GB to 0.1 GB, and each generated token takes 12–13 ms instead of 53 ms. The encoder pass, however, is not negligible: with an encoder as large as the decoder, its forward FLOPs over the context about equal those of the full-context prefill, so time to the first token is similar up to 64k tokens and lower at 110k (5.3–5.7 s against 8.9 s), where the decoder's attention over the full context grows faster than the chunked encoder. The encoder's cost does not depend on the decoder, so with a larger decoder the prefill saving would grow in proportion to its size; we did not measure that pairing. An encoding is specific to its question (Section 6), so each new question needs a new encoder pass, whereas a full-context prefill can be cached and reused across questions.

9 Agentic tasks

Agent evaluations run the full harness with at most 12 turns and 40,000 decoder tokens per episode. In the *text* arms, tool results are returned as text cut to the same per-result budget. The policy was trained on episodes with compressed tool results, so the text arms run it outside its training distribution.

Bug localisation. On SWE-bench Lite [9] the agent starts from the issue and the repository at its base commit, explores, and names the file to change. We excluded the 114 Django instances, whose repository trees exceeded our caching limit, leaving 186 (Table 8). With the repository overview, a few hundred vectors standing for the whole tree, in its context, the agent commits to an answer in 59% of episodes instead of 22%, never ends in a loop (70% without it), and uses 13.9k instead of 19.5k decoder tokens. It names a file edited by the reference patch in 14 of 186 episodes (7.5%;

Context	Arm	Decoder positions	Encoder s	Prefill s	First token s	ms per token	Memory GB	TFLOP
4,096	full context	4,143	–	0.08	0.08	5.6	0.19	3
	bgkit, 1,024	1,072	0.10	0.02	0.11	5.4	0.09	4
	bgkit, 2,048	2,096	0.10	0.04	0.13	5.6	0.11	5
16,384	full context	16,431	–	0.39	0.39	7.6	0.74	12
	bgkit, 1,024	1,078	0.38	0.02	0.39	5.4	0.09	13
	bgkit, 2,048	2,102	0.84	0.08	0.92	12.3	0.11	14
32,768	full context	32,815	–	1.89	1.89	23.4	1.48	23
	bgkit, 1,024	1,086	1.72	0.04	1.76	12.1	0.09	26
	bgkit, 2,048	2,110	0.77	0.04	0.81	5.6	0.11	26
65,536	full context	65,583	–	3.12	3.12	36.0	2.96	46
	bgkit, 1,024	1,102	3.33	0.04	3.38	11.9	0.09	51
	bgkit, 2,048	2,126	3.38	0.09	3.46	12.5	0.11	51
110,000	full context	110,047	–	8.87	8.87	52.8	4.96	78
	bgkit, 1,024	1,820	5.23	0.07	5.30	12.3	0.09	85
	bgkit, 2,048	2,142	5.65	0.09	5.74	12.7	0.11	85

Table 7: Cost of one question over a context of the given length (tokens; text from a LongMemEval-S history), released model in bf16 on one NVIDIA GB10 GPU that was shared with another job; medians of three runs. bgkit, B: the context compressed to about B decoder positions (at 110k tokens and $B = 1{,}024$ the 64× compression cap gives 1,820). First token: encoder plus prefill. ms per token: greedy decoding with the KV cache. Memory: peak CUDA memory beyond the weights. TFLOP: forward FLOPs, 2 × parameters × tokens over encoder and decoder.

95% interval about 4.5–12%), against 1 for the text agent with the same overview. The agent made no `browse` calls, so this measures the root overview alone. The overview changes how the agent works: with a compact view of the whole repository in context it commits to an answer and stops repeating itself, while choosing the right file remains hard for a 350M policy.

Arm	Gold file named	Gold file opened	Answered	Turns	Looped	Decoder tokens
bgkit + overview	0.075 (14)	0.086	0.59	3.9	0.00	13.9k
bgkit	0.027 (5)	0.118	0.22	7.1	0.70	19.5k
text + overview	0.005 (1)	0.011	0.04	2.3	0.00	21.6k
text	0.000 (0)	0.005	0.02	2.6	0.02	22.7k

Table 8: Bug localisation on SWE-bench Lite without Django (186 instances). Gold file named: the final answer names a file edited by the reference patch (count in parentheses). Answered: the episode ends with a file answer. Looped: the episode was stopped for a repeated call. Decoder tokens: per episode.

Web research. The agent searches a BM25 index, opens pages and finds passages, with every result compressed (Table 9). On HotpotQA questions searched over the pool of 87,593 validation paragraphs, judge accuracy is 0.27 against 0.14 for the text agent, with 2.4k instead of 6.5k decoder

tokens per episode and loops in 4% of episodes instead of 43%. On MuSiQue both arms are near zero (0.05 and 0.03), and on BrowseComp-Plus [2] neither answers any question.

Benchmark	Arm	Judge acc.	EM	Turns	Looped	Decoder tokens
HotpotQA (pool)	bgkit	0.27	0.16	2.7	0.04	2.4k
	text	0.14	0.11	6.1	0.43	6.5k
MuSiQue (pool)	bgkit	0.05	0.01	3.1	0.02	3.3k
	text	0.03	0.03	6.3	0.44	7.2k
BrowseComp-Plus	bgkit	0.00	0.00	2.1	0.05	3.0k
	text	0.00	0.00	2.3	0.10	3.4k

Table 9: Web research (100 questions each). HotpotQA and MuSiQue search the pooled paragraphs of their validation sets; BrowseComp-Plus searches its fixed web corpus.

Conversational memory. LongMemEval-S and LoCoMo were also run with a two-stage memory agent, which sees an index of sessions compressed under a generic prompt and re-reads sessions under its own focus. It reaches judge accuracy of 0.05 on LongMemEval-S and 0.06 on LoCoMo, against 0.30 and 0.28 when every session is compressed under the question (Table 1).

10 Limitations

- **Baselines.** No decoder was trained on the same data with text contexts at the same budget; text baselines are read by the slot-trained decoder or the original one, whichever scores higher. An adapter from an earlier, shorter run (4,000 steps on a different mix of reading and agent data) trained to read text placed in the slot scores significantly below the compressed context in 20 of 26 cells and above it in none, but it is not a matched baseline. LongLLMLingua used a 1.7B scoring model, and was run at one budget on the two longest benchmarks.
- **Task overlap.** The task formats of BABILong, HotpotQA, QuALITY and NarrativeQA are in training (their training splits, or bAbI's generator for BABILong). MuSiQue, RULER, LongMemEval and LoCoMo were not trained on.
- **Scale and sample size.** Encoder and decoder have 350M parameters each, evaluation cells have 60–186 items, and the agentic results rest on small counts. We ran one seed.
- **Encoder cost.** An encoding made under a question serves that question, so each new question needs a new encoder pass over the text. With equal-sized encoder and decoder, that pass costs about as much compute as a full-context prefill (Section 8); the savings are in decoder positions, memory and per-token decoding time.
- **Exact strings.** Arbitrary keys and numbers are lost at these ratios (RULER).
- **Training history.** The evaluated model ends a chain of warm-started development runs, and the recipe has not been reproduced from scratch.

11 Related work

Prior work compresses context for language models in several ways: summary tokens the model learns to write for itself [4, 16]; autoencoders that produce memory slots for a frozen decoder [5]; a single retrieval embedding projected into the decoder [3]; context encoders read through cross-attention [24]; and dropping tokens, either independently of the question [7, 17] or conditioned on it [8]. Question-focused abstractive compression of retrieved documents [22] corresponds to our summary baseline. Vision–language models connect separately trained encoders to language models through projections into the embedding space [1, 12]. Closest to BGKIT is recent soft compression for retrieval-augmented generation: COCOM compresses passages into a few context embeddings at adjustable rates [19], OSCAR compresses retrieved documents online and conditioned on the query [14], and SeleCom replaces full compression with a query-conditioned selector that keeps only query-relevant information [13]. BGKIT shares the idea of query-conditioned vectors read through the generator's embedding space; its particular design is the selection of content positions that are marked and then further contextualised over the whole text, a budget chosen per slot at inference time with a single checkpoint, and one interface for documents, tool output, conversation histories and hierarchical repository overviews, through which an agent receives all tool output.

12 Conclusion

BGKIT gives a language model a common interface for reading compressed context: an encoder, conditioned on a prompt that specifies the information need, produces a chosen number of vectors that the decoder reads in slots alongside ordinary text. With a 350M encoder and a 350M decoder, these vectors are far more useful than the tokens at the positions they occupy, and with the decoder held fixed the compressed context can surpass the full text. At equal budget it outperforms truncation and token pruning on most benchmarks, and BM25 retrieval on tasks that integrate facts distributed across the context; it does not retain exact strings, where retrieval remains preferable.

We see the work as a progression. *Demonstrated now*: useful question-conditioned representations under a small decoder budget, with substantially lower decoder memory and decoding time. *Available as a building block*: one interface for documents, tool results, histories and hierarchical overviews, with which an agent can keep a repository overview or memory index in context as persistent background; our evidence for this use is a 350M agent (Section 9). *Next*: background context that survives changing questions, with focused re-reading when details are needed. Because slots mix freely with text, the exact-string weakness suggests a direct extension: compressed context accompanied by a small amount of verbatim evidence for identifiers, numbers and quotations. Because the encoder's cost is independent of the decoder's size, the case for the design strengthens with larger decoders; pairing the small encoder with a much stronger decoder on a realistic reading or tool workflow is the most informative next experiment, alongside a decoder trained on text at the same budget as a matched baseline.

References

[1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, et al. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, 2022.

[2] Zijian Chen et al. BrowseComp-Plus: A more fair and transparent evaluation benchmark of deep-research agent. *arXiv preprint arXiv:2508.06600*, 2025.

[3] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. xrag: Extreme context compression for retrieval-augmented generation with one token. In *Advances in Neural Information Processing Systems*, 2024.

[4] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of EMNLP*, 2023.

[5] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations*, 2024.

[6] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Conference on Language Modeling*, 2024.

[7] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023.

[8] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of ACL*, 2024.

[9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[10] Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. The NarrativeQA reading comprehension challenge. *Transactions of the ACL*, 2018.

[11] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

[12] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2023.

[13] Yunhao Liu, Zian Jia, Xinyu Gao, Kanjun Xu, and Yun Xiong. Rethinking soft compression in retrieval-augmented generation: A query-conditioned selector perspective. In *Proceedings of the ACM Web Conference (WWW)*, 2026.

[14] Maxime Louis, Thibault Formal, Hervé Déjean, and Stéphane Clinchant. OSCAR: Online soft compression and reranking. *arXiv preprint arXiv:2504.07109*, 2025.

[15] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of ACL*, 2024.

[16] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, 2023.

[17] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, et al. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of ACL*, 2024.

[18] Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, et al. QuALITY: Question answering with long input texts, yes! In *Proceedings of NAACL*, 2022.

[19] David Rau, Shuai Wang, Hervé Déjean, and Stéphane Clinchant. Context embeddings for efficient answer generation in RAG. In *Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM)*, 2025.

[20] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the ACL*, 2022.

[21] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations*, 2025.

[22] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation. In *International Conference on Learning Representations*, 2024.

[23] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.

[24] Howard Yen, Tianyu Gao, and Danqi Chen. Long-context language modeling with parallel context encoding. In *Proceedings of ACL*, 2024.

A Paired differences

Table 10 gives, for every benchmark and budget of Table 1, the mean per-item difference between BGKIT and each other arm with its 95% paired-bootstrap interval (10,000 resamples). Text baselines use the better of the two decoders, as in Table 1.

Benchmark	Budget	bgkit minus				
		text	BM25	LongLLMLingua	summary	full context
BABILong 16k	1,024	+0.62 [+0.52, +0.72]	+0.34 [+0.24, +0.44]	+0.65 [+0.55, +0.74]	–	+0.56 [+0.45, +0.67]
	2,048	+0.57 [+0.46, +0.67]	+0.33 [+0.22, +0.44]	+0.61 [+0.51, +0.72]	–	+0.58 [+0.46, +0.69]
	4,096	+0.52 [+0.41, +0.63]	+0.29 [+0.18, +0.40]	+0.47 [+0.36, +0.58]	–	+0.56 [+0.45, +0.67]
BABILong 4k	512	+0.59 [+0.46, +0.70]	+0.40 [+0.28, +0.51]	+0.66 [+0.56, +0.75]	+0.47 [+0.35, +0.58]	+0.52 [+0.41, +0.62]
	1,024	+0.55 [+0.43, +0.66]	+0.39 [+0.28, +0.50]	+0.60 [+0.49, +0.71]	+0.56 [+0.46, +0.66]	+0.54 [+0.43, +0.64]
	2,048	+0.55 [+0.44, +0.66]	+0.50 [+0.39, +0.61]	+0.57 [+0.46, +0.68]	–	+0.61 [+0.51, +0.71]
RULER 4k	512	+0.11 [+0.03, +0.19]	−0.22 [−0.31, −0.13]	+0.18 [+0.10, +0.26]	−0.21 [−0.33, −0.10]	−0.38 [−0.48, −0.28]
	1,024	+0.05 [−0.04, +0.14]	−0.29 [−0.39, −0.19]	+0.02 [−0.08, +0.12]	−0.14 [−0.25, −0.03]	−0.38 [−0.48, −0.28]
	2,048	−0.05 [−0.15, +0.05]	−0.37 [−0.47, −0.27]	−0.22 [−0.32, −0.12]	–	−0.41 [−0.51, −0.31]
QuALITY	512	+0.05 [−0.03, +0.13]	−0.01 [−0.09, +0.07]	+0.06 [−0.02, +0.14]	−0.03 [−0.15, +0.08]	+0.05 [−0.07, +0.16]
	1,024	+0.03 [−0.06, +0.12]	+0.03 [−0.07, +0.13]	+0.08 [+0.00, +0.17]	–	+0.05 [−0.06, +0.16]
	2,048	+0.03 [−0.07, +0.13]	+0.04 [−0.06, +0.14]	+0.00 [−0.10, +0.10]	−0.07 [−0.18, +0.05]	+0.06 [−0.05, +0.17]
NarrativeQA	2,048	+0.10 [+0.02, +0.20]	+0.03 [−0.07, +0.13]	+0.05 [−0.05, +0.15]	–	–
	4,096	+0.07 [−0.02, +0.17]	+0.02 [−0.08, +0.13]	–	–	–
	8,192	+0.03 [−0.07, +0.13]	+0.02 [−0.07, +0.10]	–	–	–
HotpotQA	256	+0.41 [+0.31, +0.51]	+0.25 [+0.15, +0.36]	+0.28 [+0.15, +0.41]	+0.07 [−0.05, +0.19]	+0.19 [+0.09, +0.29]
	512	+0.27 [+0.16, +0.38]	+0.16 [+0.04, +0.28]	+0.28 [+0.15, +0.41]	+0.09 [−0.01, +0.19]	+0.17 [+0.07, +0.27]
	1,024	+0.23 [+0.13, +0.34]	+0.14 [+0.01, +0.26]	+0.19 [+0.07, +0.31]	–	+0.15 [+0.04, +0.26]
MuSiQue	512	+0.22 [+0.13, +0.32]	+0.11 [+0.03, +0.20]	+0.20 [+0.10, +0.31]	−0.13 [−0.23, −0.03]	+0.17 [+0.06, +0.27]
	1,024	+0.23 [+0.14, +0.32]	+0.14 [+0.03, +0.25]	+0.16 [+0.05, +0.27]	−0.12 [−0.24, +0.00]	+0.18 [+0.07, +0.29]
	2,048	+0.17 [+0.06, +0.28]	+0.15 [+0.04, +0.26]	+0.16 [+0.05, +0.27]	–	+0.20 [+0.09, +0.31]
LongMemEval-S	2,048	+0.12 [+0.00, +0.23]	−0.02 [−0.13, +0.10]	+0.03 [−0.10, +0.17]	–	–
	4,096	+0.08 [−0.02, +0.18]	−0.10 [−0.22, +0.02]	–	–	–
	8,192	+0.05 [−0.03, +0.13]	−0.07 [−0.18, +0.05]	–	–	–
LoCoMo	1,024	+0.19 [+0.11, +0.27]	−0.06 [−0.15, +0.03]	+0.10 [−0.01, +0.21]	–	+0.09 [−0.01, +0.19]
	2,048	+0.12 [+0.03, +0.21]	−0.02 [−0.12, +0.08]	+0.04 [−0.07, +0.15]	–	+0.09 [+0.00, +0.18]

Table 10: BGKIT minus each arm: mean per-item difference and 95% paired-bootstrap interval. Bold: the interval excludes zero. Full context: the original decoder reading the whole context (not budget-matched), on the items shared with the sweep.